Introduction to Artificial Intelligence & Expert Systems

ITE 153

Dr. Lumer Jude Doce
Faculty | IT & Physics Department

Search Algorithms in AI

Search Algorithms in AI

- Search algorithms are one of the most important areas of Artificial Intelligence. This topic will explain all about the search algorithms in AI.
- In Artificial Intelligence, Search techniques are universal problem-solving methods.
- **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation.

Search Algorithms in AI

- Search algorithms are fundamental techniques used in computer science and artificial intelligence to find solutions or explore possibilities in various problem domains.
- These algorithms operate by systematically examining a search space, which is a set of states or configurations, to find a desired goal state or solution.

Search Algorithms in AI

Search Algorithm Terminologies

- **Search:** Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:
- **Search Space:** Search space represents a set of possible solutions, which a system may have.
    - **Start State:** It is a state from where agent begins **the search**.
    - **Goal test:** It is a function which observe the current state and returns whether the goal state is achieved or not.
- **Search tree:** A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.

Search Algorithms in AI

Search Algorithm Terminologies

- **Actions:** It gives the description of all the available actions to the agent.
- **Transition model:** A description of what each action do, can be represented as a transition model.
- **Path Cost:** It is a function which assigns a numeric cost to each path.
- **Solution:** It is an action sequence which leads from the start node to the goal node.
- **Optimal Solution:** If a solution has the lowest cost among all solutions.

Search Algorithms in AI

Properties of Search Algorithms

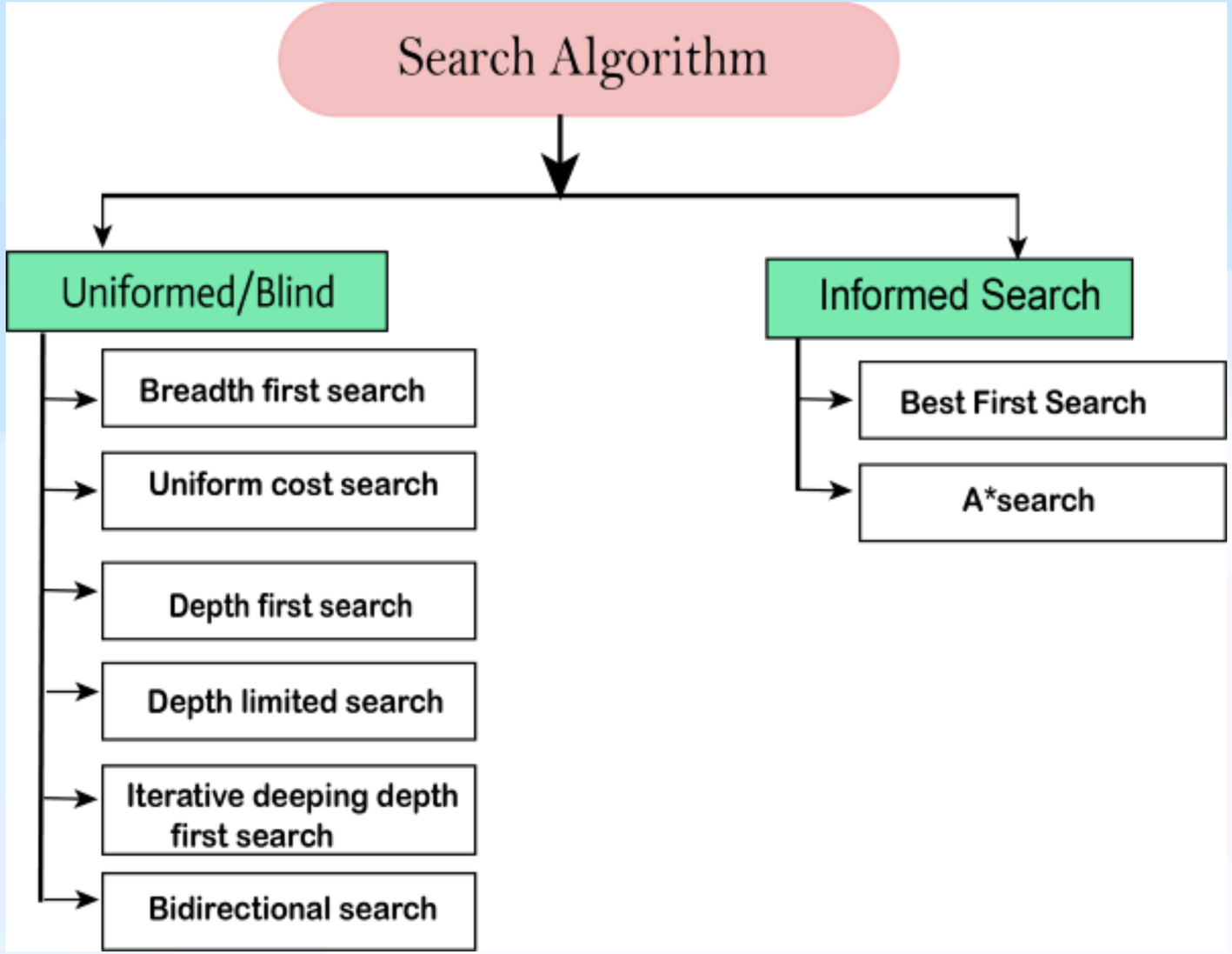
- Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:
  - **Completeness:** A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.
  - **Optimality:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.
  - **Time Complexity:** Time complexity is a measure of time for an algorithm to complete its task.
  - **Space Complexity:** It is the maximum storage space required at any point during the search, as the complexity of the problem.

Types of Search Algorithms in AI

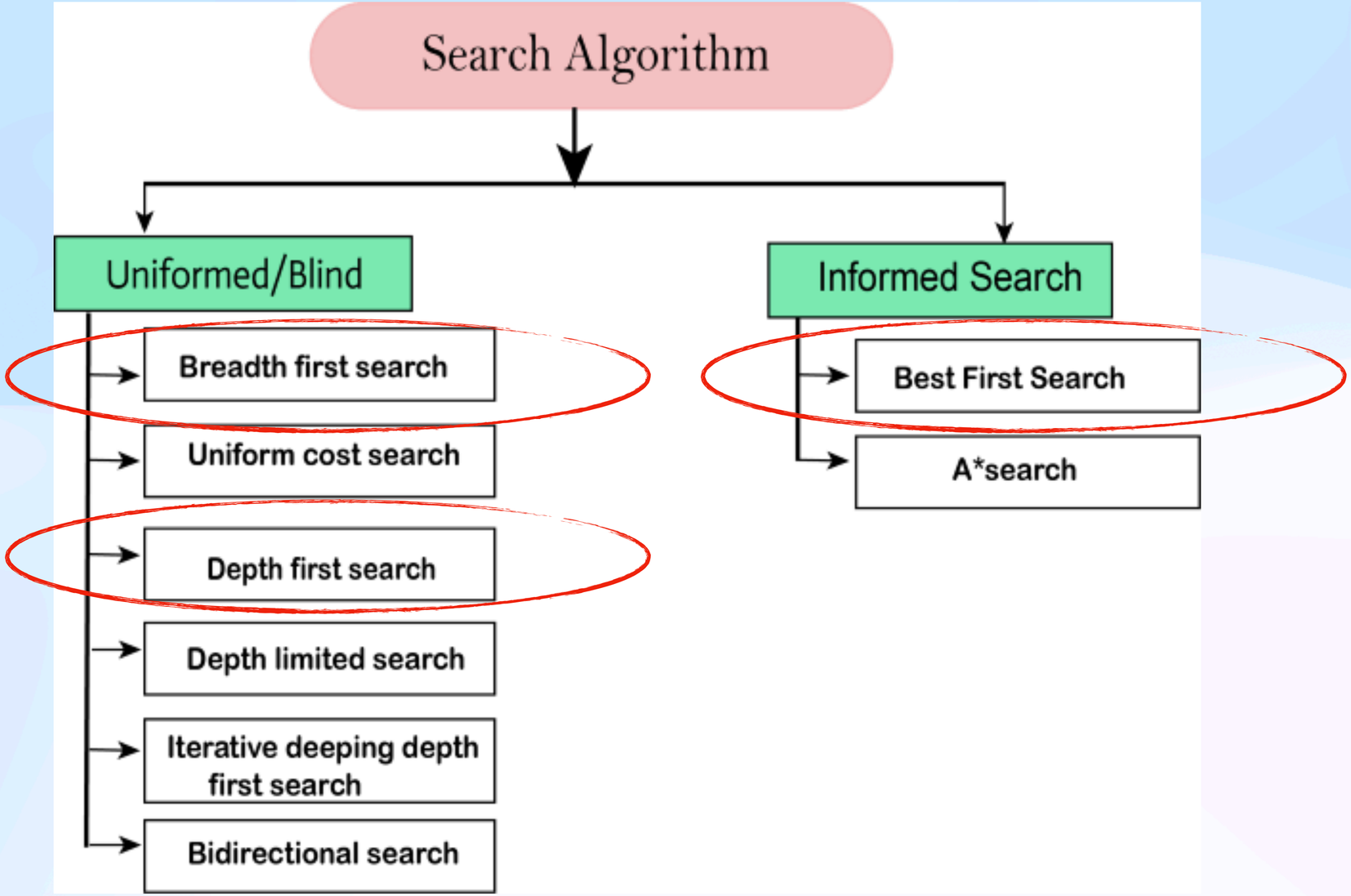
Types of Search Algorithms in AI

- Based on the search problems, we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.

Types of Search Algorithms in AI



Types of Search Algorithms in AI



Uninformed/Blind Search

Uninformed/Blind Search

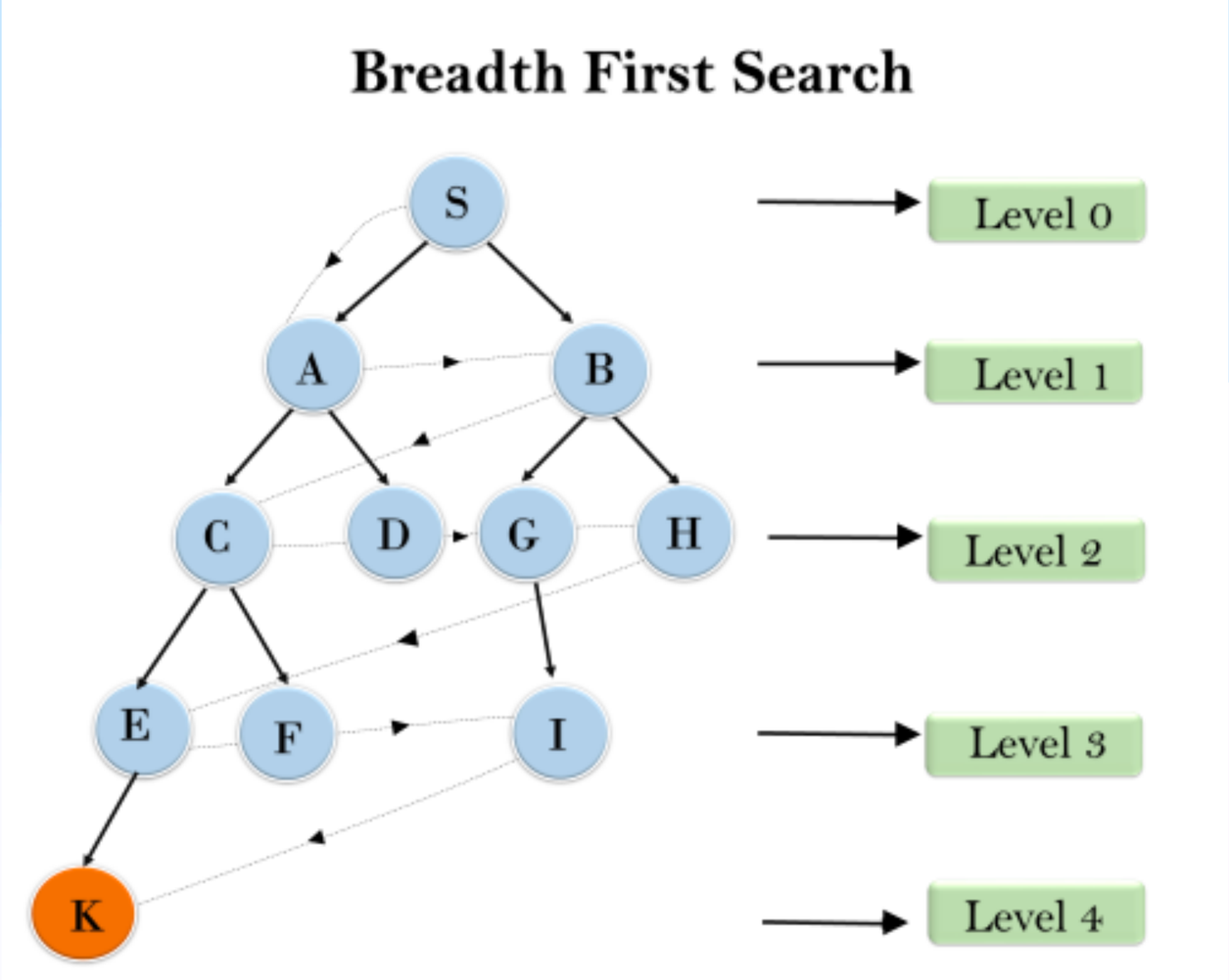
Breadth-first Search

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Uninformed/Blind Search

Breadth-first Search

- Traversing of the tree using BFS algorithm from the root node S to goal node K.
- BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:
    - S---> A--->B---->C--->D---->G--->H--->E---->F---->I---->K



Uninformed/Blind Search

Breadth-first Search

- **Time Complexity:** Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.
- $T(b) = 1+b^2+b^3+.....+ b^d= O(b^d)$
- **Space Complexity:** Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$
- **Completeness:** BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.
- **Optimality:** BFS is optimal if path cost is a non-decreasing function of the depth of the node.

Uninformed/Blind Search

Depth-first Search

- Depth-first search isa recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Uninformed/Blind Search

Depth-first Search

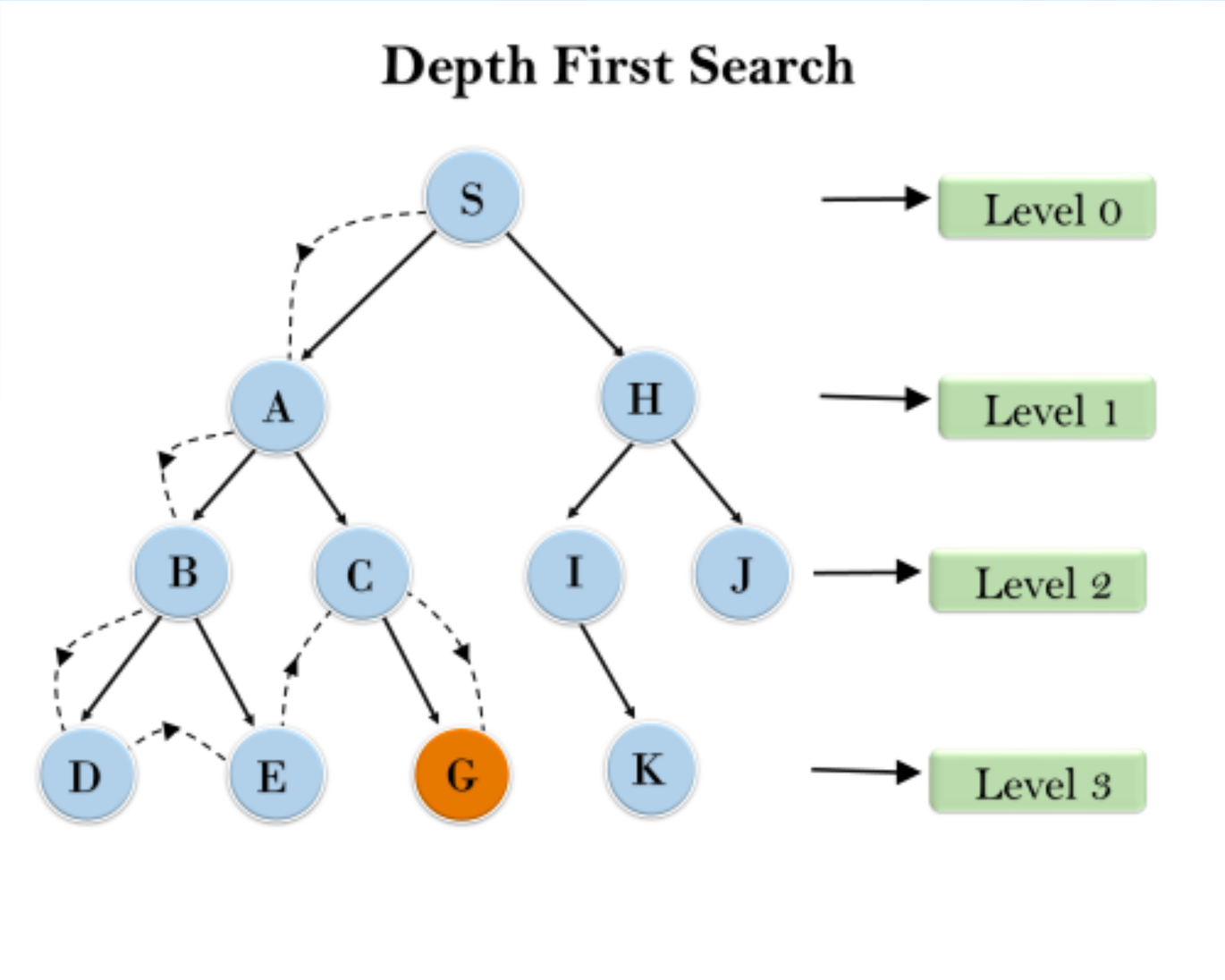
- Advantage:
  - DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
  - It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).
- Disadvantage:
  - There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
  - DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Uninformed/Blind Search

Depth-first Search

- The flow of depth-first search, and it will follow the order as: Root node--->Left node ----> right node.

- It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Uninformed/Blind Search

Depth-first Search

- **Completeness:** DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.
- **Time Complexity:** Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by: $T(n)= 1+ n^2+ n^3 +.....+ n^m=O(n^m)$ Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)
- **Space Complexity:** DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.
- **Optimal:** DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

Informed Search

Informed Search

Heuristic

- The uninformed search algorithms looked through search space for all possible solutions of the problem without having any additional knowledge about search space.
- Informed search algorithm contains an array of knowledge such as how far we are from the goal, path cost, how to reach to goal node, etc. This knowledge help agents to explore less to the search space and find more efficiently the goal node.
- The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Informed Search

Heuristic

- **Heuristics function:** Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.
- The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by h(n), and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

Informed Search

Heuristic

- Admissibility of the heuristic function is given as:

  $h(n) <= h^*(n)$

- Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Informed Search

Heuristic

- Pure heuristic search is the simplest form of heuristic search algorithms.
- It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.
- On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues unit a goal state is found.

Informed Search

Best-first Search Algorithm (Greedy Search)

- Greedy best-first search algorithm always selects the path which appears best at that moment.
- It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search.
- Best-first search allows us to take the advantages of both algorithms.

Informed Search

Best-first Search Algorithm (Greedy Search)

- With the help of best-first search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

  $$f(n) = g(n).$$

  where, $g(n)$= estimated cost from node n to the goal.

- The greedy best first algorithm is implemented by the priority queue.

Informed Search

Best-first Search Algorithm (Greedy Search)

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
- **Step 4:** Expand the node n, and generate the successors of node n.
- **Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

Informed Search

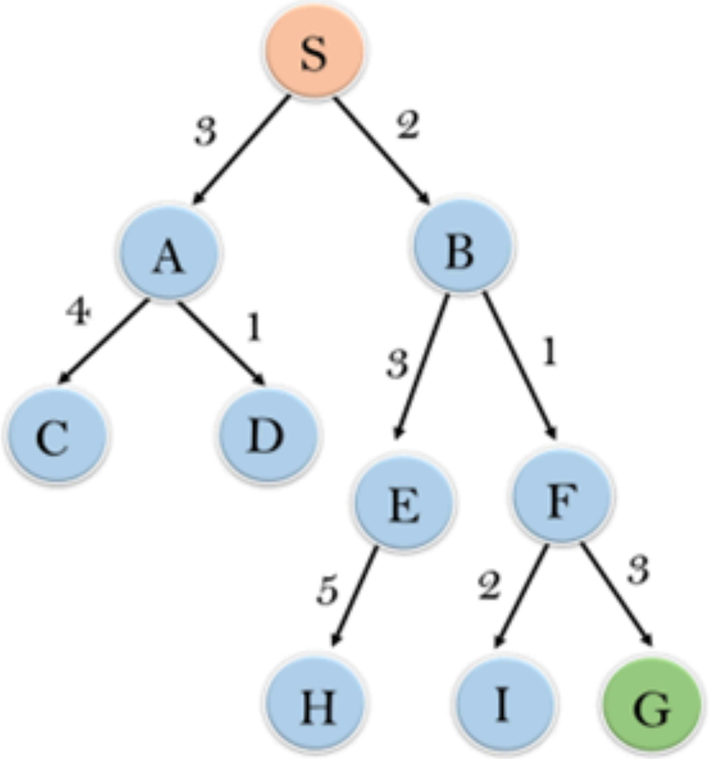
Best-first Search Algorithm (Greedy Search)

- Advantages:
  - Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
  - This algorithm is more efficient than BFS and DFS algorithms.
- Disadvantages:
  - It can behave as an unguided depth-first search in the worst case scenario.
  - It can get stuck in a loop as DFS.
  - This algorithm is not optimal.

Informed Search

Best-first Search Algorithm (Greedy Search)

- Consider the search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the table.

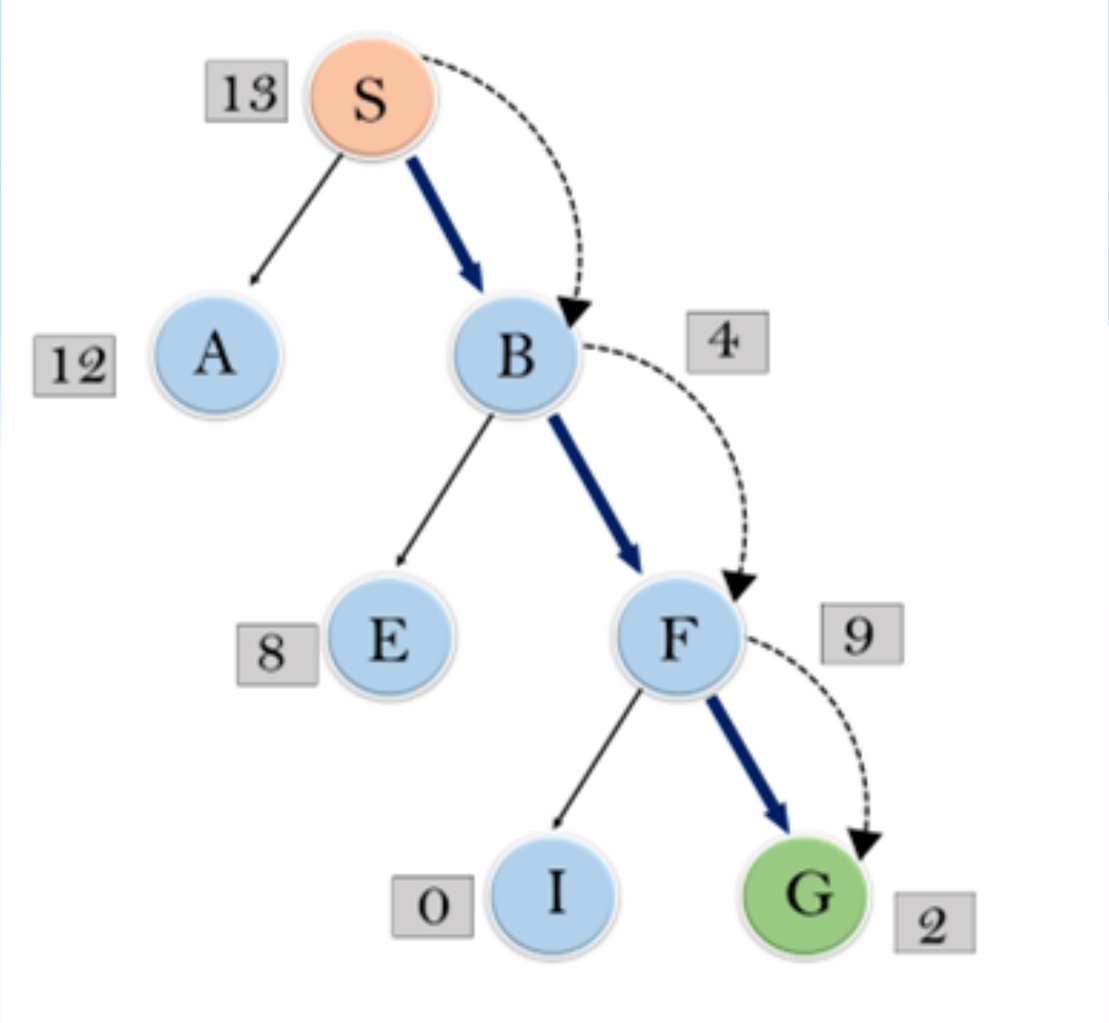


node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Informed Search

Best-first Search Algorithm (Greedy Search)

- In this search example, we are using two lists which
  are **OPEN** and **CLOSED** Lists.
- Following are the iteration for traversing the example in the right.

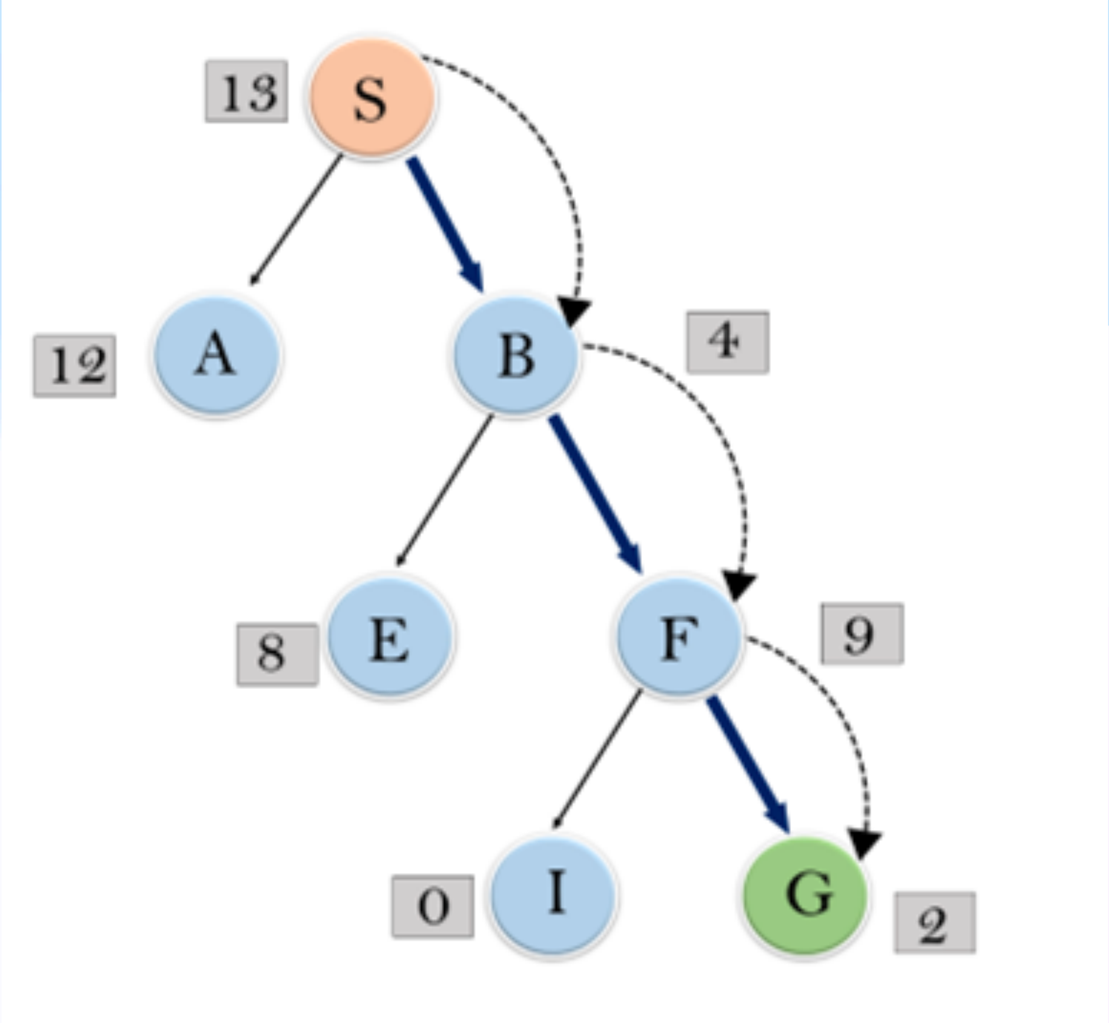


Informed Search

Best-first Search Algorithm (Greedy Search)

- Expand the nodes of S and put in the CLOSED list
- **Initialization:** Open [A, B], Closed [S]
- **Iteration 1:** Open [A], Closed [S, B]
- **Iteration 2:** Open [E, F, A], Closed [S, B]
  : Open [E, A], Closed [S, B, F]
- **Iteration 3:** Open [I, G, E, A], Closed [S, B, F]
  : Open [I, E, A], Closed [S, B, F, G]
- Hence the final solution path will be:

  S----> B----->F----> G



Informed Search

Best-first Search Algorithm (Greedy Search)

- **Time Complexity:** The worst case time complexity of Greedy best first search is $O(b^m)$.
- **Space Complexity:** The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.
- **Complete:** Greedy best-first search is also incomplete, even if the given state space is finite.
- **Optimal:** Greedy best first search algorithm is not optimal.